Sintaxis Strudel Limpio

1. Introducción a Strudel

Strudel es un entorno de *live coding* musical que permite crear y modificar música escribiendo código. En lugar de producir música únicamente mediante una interfaz gráfica, se describen patrones mediante una sintaxis textual y funciones de JavaScript.

Su sintaxis permite expresar ritmos, notas, acordes, repeticiones, silencios, efectos y transformaciones temporales de forma compacta.

Para nuestro TFG, Strudel es una referencia para investigar cómo diseñar un lenguaje que permita escribir patrones musicales y transformarlos antes de enviar las instrucciones al motor de audio o a Reaper.

1.1. Conceptos básicos

En Strudel, un **patrón** describe eventos musicales distribuidos en el tiempo. Cada evento puede tener diferentes parámetros que determinan cómo se reproduce.

Concepto o parámetro	Significado
`note`	Permite especificar notas mediante nombres musicales, como `c4`, `e4` o `g4`.
`n`	Permite especificar valores numéricos que pueden interpretarse como grados de una escala.
`s`	Selecciona el sonido o *sample* que se reproduce.
`gain`	Controla el nivel de volumen.
`cutoff`	Controla la frecuencia de corte de un filtro.
`freq`	Permite especificar la frecuencia del sonido en hercios.
`scale()`	Permite interpretar grados numéricos dentro de una escala musical.

No todos estos elementos tienen la misma función: algunos generan patrones de eventos y otros modifican los parámetros de esos eventos.

Por ejemplo:

```
note("c4 e4 g4").s("piano")
```

Este código define una secuencia de tres notas y utiliza el sonido de piano para reproducirlas.

En cambio:

```
n("0 2 4").scale("C:major")
```

Aquí se utilizan números que representan grados de la escala de Do mayor. La función `scale()` convierte esos grados en notas musicales.

Idea fundamental: un patrón describe qué eventos se producen y cuándo; los parámetros determinan cómo suenan.

---

2. Mini Notation

Documentación de Mini Notation

La *Mini Notation* es una sintaxis compacta diseñada para describir patrones rítmicos y musicales utilizando poco texto.

Permite expresar secuencias, silencios, repeticiones, subdivisiones, acordes y probabilidades sin tener que escribir una función de JavaScript para cada evento.

2.1. Elementos básicos de la sintaxis

Elemento	Función	Ejemplo	Interpretación
Espacio	Separa eventos consecutivos.	`note("c4 e4 g4")`	Tres eventos distribuidos dentro del ciclo.
`~`	Introduce un silencio.	`s("bd ~ bd ~")`	Alterna golpes de bombo y silencios.
`*n`	Acelera o repite un patrón dentro del mismo intervalo temporal.	`note("[e5 b4 d5 c5]*2")`	Reproduce la secuencia dos veces en el tiempo que ocuparía una vez.
`/n`	Alarga el patrón para que ocupe más ciclos.	`note("[e5 b4 d5 c5]/2")`	Distribuye la secuencia a lo largo de dos ciclos.
`< >`	Alterna entre diferentes elementos según el ciclo.	`note("<c4 e4 g4>")`	Selecciona un elemento distinto en cada ciclo.
`[ ]`	Agrupa eventos para tratarlos como una unidad dentro del patrón.	`note("c4 [e4 g4] b4")`	Agrupa las notas interiores y las distribuye dentro del espacio asignado al grupo.
`,`	Combina eventos simultáneos.	`note("g3,b3,e4")`	Reproduce las tres notas como un acorde.
`@n`	Asigna un peso temporal a un evento o grupo.	`note("<[c4 e4]@2 [g4 b4]>")`	El primer grupo recibe un peso temporal mayor.
`_`	Permite alargar un evento mediante la notación de duración.	`note("c4 _ _ e4")`	Se utiliza para representar una nota sostenida a través de subdivisiones.
`!n`	Repite un evento un número determinado de veces sin acelerar el patrón como haría `*n`.	`note("c4!2 e4")`	Repite el primer evento dos veces.

Elemento	Función	Ejemplo	Interpretación
`?`	Introduce una condición de probabilidad para que un evento aparezca o no.	`s("bd*8?")`	Cada evento tiene una probabilidad de aparecer.
`?p`	Permite especificar la probabilidad de aparición.	`s("bd*8?0.1")`	Utiliza una probabilidad de 0,1 para cada evento condicionado.
`|`	Permite expresar alternativas entre patrones.	`note("c4 | e4 | g4")`	Permite seleccionar entre distintas alternativas según las reglas de la notación.

Importante: los símbolos modifican cómo se distribuyen los eventos en el tiempo. No todos crean eventos nuevos: algunos agrupan, otros repiten y otros cambian su duración o probabilidad.

La duración real de un ciclo depende de la configuración temporal de Strudel. Un ciclo no equivale necesariamente a un segundo.

2.2. Agrupación y acordes

Los corchetes permiten agrupar eventos, mientras que las comas permiten reproducir notas simultáneamente.

Ejemplo de una secuencia de acordes:

```
note("<[g3,b3,e4] [a3,c4,e4] [b3,d4,f#4] [b3,e4,g4]>*2")
```

La expresión alterna entre cuatro acordes y repite la secuencia dos veces dentro del intervalo temporal correspondiente.

La agrupación permite construir estructuras más complejas sin escribir cada nota como un patrón independiente.

2.3. Ritmos euclídeos

Los ritmos euclídeos distribuyen un número determinado de pulsos de la manera más uniforme posible dentro de un número de posiciones.

La notación general es:

```
s("bd(3,8)")
```

Parámetro	Significado
Beats	Número de pulsos que se van a distribuir.
Segments	Número total de posiciones disponibles.
Offset	Desplazamiento opcional de la distribución rítmica.

Ejemplo con un ritmo adicional:

```
s("bd(3,8,0), hh cp")
```

El bombo utiliza un ritmo euclídeo de tres pulsos en ocho posiciones. Los demás elementos se combinan con ese patrón.

Los ritmos euclídeos permiten generar patrones rítmicos interesantes sin tener que especificar manualmente la posición de cada golpe.

---

3. JavaScript API: construir patrones mediante funciones

Documentación de Pattern Factories

La JavaScript API permite construir patrones mediante funciones, en lugar de depender exclusivamente de la Mini Notation.

Esto facilita combinar patrones, reutilizar estructuras y crear abstracciones propias.

3.1. Equivalencia entre Mini Notation y funciones

Función	Equivalencia aproximada en Mini Notation	Descripción
`cat(x, y)`	`"<x y>"`	Concatena patrones, asignando un ciclo a cada elemento.
`seq(x, y)`	`"x y"`	Distribuye los elementos secuencialmente dentro de un ciclo.
`stack(x, y)`	`"x,y"`	Superpone patrones para que suenen simultáneamente.
`stepcat([3,x], [2,y])`	`"x@3 y@2"`	Concatena patrones asignando pesos temporales a cada elemento.
`polymeter([a,b,c], [x,y])`	`"{a b c, x y}"`	Combina patrones con estructuras métricas diferentes.
`silence`	`"~"`	Representa un patrón silencioso.
`run(n)`	Secuencia de `0` a `n-1`	Genera una secuencia de números consecutivos.
`binary(n)`	Patrón binario	Genera una estructura rítmica a partir de la representación binaria de un número.
`binaryN(n, bits)`	Patrón binario de longitud definida	Genera una representación binaria con un número de bits especificado.

Las equivalencias son orientativas: la interpretación exacta depende del contexto y de cómo se construye el patrón.

3.2. Ejemplo: generar una secuencia de notas

Con Mini Notation:

```
note("c3 eb3 g3")
```

Mediante funciones:

```
note(seq("c3", "eb3", "g3"))
```

Ambas expresiones permiten definir una secuencia de notas, pero la segunda hace explícita la construcción del patrón mediante la función `seq()`.

3.3. Combinar patrones

Un patrón puede construirse mediante varias funciones equivalentes.
Primera forma: concatenar acordes.

```
cat(
  "g3,b3,e4",
  "a3,c4,e4",
  "b3,d4,f#4",
  "b3,e4,g4"
).note()
```

Segunda forma: construir cada acorde mediante `**stack()**`.

```
cat(
  stack("g3", "b3", "e4"),
  stack("a3", "c4", "e4"),
  stack("b3", "d4", "f#4"),
  stack("b3", "e4", "g4")
).note()
```

Tercera forma: utilizar Mini Notation.

```
note("<[g3,b3,e4] [a3,c4,e4] [b3,d4,f#4] [b3,e4,g4]>")
```

Las tres expresiones representan una progresión de cuatro acordes mediante mecanismos diferentes.

La primera utiliza cadenas con acordes; la segunda construye los acordes con funciones; la tercera utiliza la sintaxis compacta de Mini Notation.

3.4. Mezclar ritmos individuales

Para combinar patrones rítmicos diferentes puede ser más cómodo utilizar funciones como `stack()`.

```
stack(
  note("c2 eb2(3,8)")
    .s("sawtooth")
    .cutoff(800),
```

```
  s("bd(5,8), hh*8")
)
```

En este ejemplo se superponen dos capas:

- Una secuencia de notas graves con sonido de diente de sierra y filtro.
- Una capa rítmica de bombo y hi-hat.

Cada capa conserva su propia estructura y puede tener sus propios parámetros.

---

4. Value Modifiers: modificar los valores de los patrones

Documentación de Value Modifiers

Los *value modifiers* son funciones que transforman los valores de los eventos de un patrón.

En lugar de reescribir los datos, permiten aplicar operaciones matemáticas, redondear números, transformar rangos o interpretar proporciones.

Por ejemplo:

```
n("0 2 4".add(3)).scale("C:major")
```

Primero se suma tres a cada valor. Después, los nuevos valores se interpretan como grados de la escala de Do mayor.

4.1. Operaciones matemáticas

Función	Descripción	Ejemplo en Strudel	Resultado esperado
`add(valor)`	Suma una cantidad a cada valor.	`n("0 2 4".add(3)).scale("C:major")`	Los grados pasan a ser `3 5 7`.
`sub(valor)`	Resta una cantidad a cada valor.	`n("0 2 4".sub(1)).scale("C:major")`	Los grados pasan a ser `-1 1 3`.
`mul(valor)`	Multiplica cada valor por un factor.	`"<1 2 3>".mul(150).freq()`	Utiliza los valores multiplicados por 150 como frecuencias.
`div(valor)`	Divide cada valor por una cantidad.	`"<200 400 800>".div(2).freq()`	Utiliza las frecuencias `100`, `200` y `400` Hz.

4.2. Funciones de redondeo

Función	Descripción	Ejemplo en Strudel	Interpretación
`round()`	Redondea al entero más	`n("0.5 1.5 2.5".round()).scale("C:major")`	Redondea los valores antes de interpretarlos

Función	Descripción	Ejemplo en Strudel	Interpretación
	cercano.		como grados.
`floor()`	Redondea hacia abajo.	`note("42 42.1 42.5 43".floor())`	Produce los valores `42 42 42 43`.
`ceil()`	Redondea hacia arriba.	`note("42 42.1 42.5 43".ceil())`	Produce los valores `42 43 43 43`.

4.3. Transformar rangos

Estas funciones son útiles para controlar parámetros cuyo valor cambia a lo largo del tiempo.

Función	Descripción	Ejemplo en Strudel	Aplicación
`range(min, max)`	Mapea una señal de 0 a 1 a un rango lineal.	`s("bd*4").cutoff(sine.range(500, 4000))`	Automatiza el filtro entre 500 y 4000 Hz.
`rangex(min, max)`	Mapea una señal de 0 a 1 mediante una curva exponencial.	`s("bd*4").cutoff(sine.rangex(500, 4000))`	Automatiza el filtro con una distribución exponencial.
`range2(min, max)`	Mapea una señal de -1 a 1 al rango indicado.	`s("bd*4").cutoff(sine2.range2(500, 4000))`	Permite utilizar una señal bipolar para controlar el filtro.

4.4. Otras funciones

Función	Descripción	Ejemplo en Strudel	Aplicación
`ratio()`	Permite trabajar con proporciones numéricas.	`ratio("1, 5:4, 3:2").mul(110).freq().s("piano")`	Genera frecuencias a partir de proporciones.
`as()`	Permite interpretar valores etiquetados como parámetros.	`"c:.5 a:1 f:.25 e:.8".as("note:clip")`	Representa varios valores asociados a parámetros.
Parámetros personalizados	Permiten definir nuevos controles.	Depende de la definición del parámetro.	Amplían las propiedades que se pueden controlar.

4.5. Encadenamiento de funciones

En Strudel es posible encadenar funciones para transformar un patrón y configurar su sonido.

```
note("a3 c#4 e4 a4")
  .s("sawtooth")
  .cutoff(500)
  .room(0.5)
```

El patrón se transforma progresivamente:

1. `note()` define las notas.
2. `s()` selecciona el sonido.
3. `cutoff()` modifica el filtro.
4. `room()` añade reverberación.

Las funciones pueden combinarse para construir expresiones musicales complejas a partir de una secuencia sencilla.

4.6. Crear funciones propias con `register()`

Strudel permite registrar funciones reutilizables.

```
const effectChain = register('effectChain', (pat) => pat
  .s("sawtooth")
  .cutoff(500)
  .room(0.5)
)

note("a3 c#4 e4 a4").effectChain()
```

En este ejemplo, `effectChain()` agrupa varias operaciones en una sola función.

En vez de repetir la misma cadena de efectos, se define una vez y se reutiliza en otros patrones.

Esta idea es especialmente interesante para el TFG: permite diseñar abstracciones que simplifiquen la sintaxis del usuario sin tener que repetir todas las operaciones internas.

---

5. Modificadores de tiempo

Documentación de Time Modifiers

Los modificadores de tiempo permiten alterar la distribución temporal de un patrón sin tener que reescribir sus eventos.
Se pueden utilizar para acelerar, ralentizar, desplazar, invertir, repetir o seleccionar partes de una secuencia.

5.1. Velocidad y ritmo

Función	Equivalencia en Mini Notation	Descripción	Ejemplo
`slow(n)`	`/n`	Alarga el patrón para que ocupe más tiempo.	`"bd sd".slow(2)`
`fast(n)`	`*n`	Acelera el patrón y aumenta el número de repeticiones en el intervalo temporal.	`"bd sd".fast(2)`
`euclid(k,n)`	`(k,n)`	Distribuye `k` pulsos en `n` posiciones.	`"bd".euclid(3,8)`
`euclidRot(k,n,r)`	`(k,n,r)`	Aplica un ritmo euclídeo y desplaza su distribución.	`"bd".euclidRot(3,8,1)`
`euclidLegato(k,n)`	—	Genera un ritmo euclídeo en el que los eventos se mantienen hasta el siguiente.	`"bd".euclidLegato(3,8)`
`cpm(n)`	—	Establece la velocidad en ciclos por minuto.	`"bd sd".cpm(90)`

Los ritmos euclídeos distribuyen pulsos de manera uniforme; no se limitan a cambiar la velocidad de reproducción.

5.2. Desplazamiento temporal

Función	Descripción	Ejemplo
`early(n)`	Adelanta temporalmente los eventos.	`"bd sd".early(0.1)`
`late(n)`	Retrasa temporalmente los eventos.	`"bd sd".late(0.1)`
`swingBy(amount, n)`	Aplica swing a subdivisiones del patrón.	`"bd hh".swingBy(1/3, 4)`
`swing(n)`	Abreviatura de un swing con una cantidad de referencia de un tercio.	`"bd hh".swing(4)`

Estos modificadores permiten cambiar el *groove* sin cambiar necesariamente las notas o los sonidos.

5.3. Invertir y reorganizar patrones

Función	Descripción	Ejemplo
`rev()`	Invierte el orden temporal del patrón.	`"c4 e4 g4".rev()`
`palindrome()`	Alterna el patrón con su versión invertida.	`"c4 e4 g4".palindrome()`
`iter(n)`	Divide el patrón en subdivisiones y cambia progresivamente el punto de inicio.	`"bd sd hh cp".iter(4)`

Función	Descripción	Ejemplo
`iterBack(n)`	Aplica una transformación similar a `iter()`, recorriendo las subdivisiones en sentido inverso.	`"bd sd hh cp".iterBack(4)`
`ply(n)`	Repite cada evento el número de veces indicado.	`"bd sd".ply(2)`
`fastGap(n)`	Acelera el patrón dejando un espacio vacío en el resto del ciclo.	`"bd sd".fastGap(2)`

Estas funciones permiten generar variaciones de un patrón existente sin tener que construir una secuencia completamente nueva.

5.4. Seleccionar y comprimir partes de un patrón

Función	Descripción	Ejemplo
`segment(n)`	Muestrea un patrón a un número de eventos por ciclo. Resulta útil para convertir señales continuas en eventos discretos.	`note(sine.range(40,52).segment(24))`
`compress(start,end)`	Comprime el patrón para que ocupe un intervalo temporal determinado.	`s("bd sd").compress(0.25,0.75)`
`zoom(start,end)`	Selecciona una parte del patrón y la reproduce dentro del intervalo temporal original.	`"bd sd hh cp".zoom(0.25,0.75)`
`linger(fraction)`	Selecciona una fracción del patrón y la prolonga para ocupar el intervalo correspondiente.	`"bd sd".linger(0.5)`
`clip(duration)`	Controla la duración de reproducción de los eventos o samples.	`"bd sd".clip(0.5)`
`ribbon(offset,duration)`	Permite repetir o mantener una porción del patrón mediante un desplazamiento temporal.	`s("bd!16?").ribbon(29,0.5)`

Los parámetros exactos y el comportamiento de algunas de estas funciones deben comprobarse con la documentación y los ejemplos oficiales antes de incorporarlos como especificaciones del proyecto.

5.5. Transformaciones dentro y fuera de un ciclo

Las funciones `inside()` y `outside()` permiten aplicar transformaciones temporales a distintas escalas.

```
"0 1 2 3 4 3 2 1"
  .inside(4, rev)
  .scale("C:major")
  .note()
```

Una transformación equivalente, expresada mediante modificadores de tiempo, es:

```
"0 1 2 3 4 3 2 1"
  .slow(4)
  .rev()
  .fast(4)
  .scale("C:major")
  .note()
```

La idea es que una operación puede aplicarse a una parte temporal del patrón o a una escala temporal más amplia.

---

6. Señales (Signals)

Documentación de Signals

Las **señales** (*signals*) son patrones que generan valores continuos, es decir, valores que pueden cambiar a lo largo del tiempo y que, teóricamente, tienen infinitos pasos.
A diferencia de una secuencia normal, que contiene eventos definidos en posiciones concretas, una señal genera un flujo de valores numéricos que puede utilizarse para modificar parámetros musicales.

Por ejemplo, podemos utilizar una señal para cambiar continuamente la frecuencia de corte de un filtro, el volumen o la altura de las notas.

6.1. Tipos de señales

Strudel incluye diferentes señales que generan valores siguiendo distintas formas de onda.

Señal	Descripción	Rango de valores	Ejemplo en Strudel
`saw`	Genera una onda de diente de sierra.	0 a 1	`n(saw.range(0,8).segment(8)).scale("C:major")`
`sine`	Genera una onda sinusoidal.	0 a 1	`n(sine.segment(16).range(0,15)).scale("C:minor")`
`cosine`	Genera una onda cosenoidal.	0 a 1	`n(cosine.segment(16).range(0,15)).scale("C:minor")`
`tri`	Genera una onda triangular.	0 a 1	`n(tri.segment(8).range(0,7)).scale("C:minor")`
`square`	Genera una onda cuadrada,	0 a 1	`n(square.segment(4).range(0,7)).scale("C:minor")`

Señal	Descripción	Rango de valores	Ejemplo en Strudel
	alternando entre valores.		
`rand`	Genera valores aleatorios continuos.	0 a 1	`s("bd*4,hh*8").cutoff(rand.range(500,8000))`
`perlin`	Genera ruido Perlin, con variaciones aleatorias más suaves.	0 a 1	`s("bd*4,hh*8").cutoff(perlin.range(500,8000))`

Las señales permiten generar cambios automáticos sin tener que escribir manualmente todos los valores.

6.2. Señales con valores entre -1 y 1

También existen versiones de algunas señales que utilizan un rango bipolar, es decir, generan valores entre -1 y 1.

Señal	Descripción
`saw2`	Versión bipolar de la onda de diente de sierra.
`sine2`	Versión bipolar de la onda sinusoidal.
`cosine2`	Versión bipolar de la onda cosenoidal.
`tri2`	Versión bipolar de la onda triangular.
`square2`	Versión bipolar de la onda cuadrada.
`rand2`	Genera valores aleatorios entre -1 y 1.

6.3. Señales aleatorias y binarias

Además de las señales anteriores, existen funciones para generar números enteros aleatorios o valores binarios.

Función	Descripción	Ejemplo en Strudel	Aplicación
`irand(n)`	Genera una señal de enteros aleatorios entre 0 y n-1. El valor máximo no está incluido.	`n(irand(8)).scale("C:minor")`	Selecciona aleatoriamente grados de una escala.
`brand`	Genera una señal binaria aleatoria, cuyos valores son 0 o 1.	`s("hh*10").pan(brand)`	Alterna aleatoriamente entre los valores extremos de un parámetro.

Función	Descripción	Ejemplo en Strudel	Aplicación
`brandBy(p)`	Genera valores 0 o 1, controlando la probabilidad de obtener 1.	`s("hh*10").pan(brandBy(0.2))`	Permite controlar la frecuencia con la que aparece el valor 1.

En `brandBy()`, el parámetro representa una probabilidad entre 0 y 1. Por ejemplo, `brandBy(0.2)` asigna una probabilidad del 20 % al valor 1.

6.4. Convertir señales continuas en eventos

Las señales producen valores continuamente, pero en ocasiones necesitamos convertirlos en una cantidad concreta de eventos.
Para ello podemos utilizar `segment(n)`, que muestrea la señal y genera un número determinado de valores por ciclo.

```
n(sine.segment(16).range(0,15))
  .scale("C:minor")
  .s("piano")
```

En este ejemplo:

1. `sine` genera una señal sinusoidal.
2. `segment(16)` toma 16 valores de esa señal por ciclo.
3. `range(0,15)` transforma los valores para que estén entre 0 y 15.
4. `scale("C:minor")` interpreta esos números como grados de la escala de Do menor.
5. `s("piano")` utiliza el sonido de piano.

6.5. Crear señales propias

También es posible crear señales personalizadas mediante `signal()`.

Esta función recibe otra función que determina cómo se calcula el valor de la señal a partir del tiempo.

```
signal(t => t % 1)
  .segment(16)
  .range(30,40)
  .note()
```

Otro ejemplo:

```
signal(t => Math.pow(t,2) % 1)
  .range(40,60)
  .add(5)
  .segment(16)
  .note()
```

Esto permite definir comportamientos que no están incluidos en las señales predefinidas.

Idea fundamental: las señales permiten generar valores que evolucionan automáticamente con el tiempo. Son especialmente útiles para crear modulaciones y automatizar parámetros sin tener que definir manualmente cada cambio.

---

7. Random Modifiers

Documentación de Random Modifiers

Los *random modifiers* permiten introducir aleatoriedad en los patrones musicales.
Se pueden utilizar para seleccionar sonidos al azar, eliminar eventos de una secuencia o aplicar transformaciones con una probabilidad determinada.
Esto permite generar variaciones musicales sin tener que escribir manualmente cada versión del patrón.

7.1. Seleccionar elementos aleatoriamente

Función	Descripción	Ejemplo en Strudel	Aplicación
`choose()`	Selecciona aleatoriamente uno de los elementos de una lista.	`note("c2 g2!2 d2 f1").s(choose("sine","triangle","bd:6"))`	Alterna aleatoriamente entre diferentes sonidos.
`wchoose()`	Selecciona aleatoriamente un elemento de una lista asignando un peso a cada opción.	`note("c2 g2!2 d2 f1").s(wchoose(["sine",10],["triangle",1],["bd:6",1]))`	Permite que algunos sonidos tengan más posibilidades de aparecer que otros.

En `wchoose()`, cada elemento se acompaña de un peso. Un peso mayor significa que ese elemento tiene más posibilidades de ser seleccionado.

Por ejemplo:

```
note("c2 g2 d2 f1")
  .s(wchoose(
    ["sine",10],
    ["triangle",1],
    ["bd:6",1]
  ))
```

En este caso, el sonido `sine` tiene un peso mucho mayor que los otros dos, por lo que aparecerá con más frecuencia.

7.2. Seleccionar patrones aleatorios por ciclo

Estas funciones permiten elegir un patrón completo aleatoriamente en cada ciclo.

Función	Sinónimo	Descripción	Ejemplo en Strudel
`chooseCycles()`	`randcat()`	Selecciona aleatoriamente uno de los patrones en cada ciclo.	`chooseCycles("bd","hh","sd").s().fast(8)`
`wchooseCycles()`	`wrandcat()`	Selecciona un patrón en cada ciclo utilizando pesos para controlar su probabilidad.	`wchooseCycles(["bd",10],["hh",1],["sd",1]).s().fast(8)`

También se pueden utilizar directamente alternativas mediante Mini Notation:

```
s("bd | hh | sd").fast(8)
```

En el caso de `wchooseCycles()`, los pesos pueden incluso estar definidos mediante otro patrón:

```
wchooseCycles(
  ["bd(3,8)", "<5 0>"],
  ["hh hh hh", 3]
).fast(4).s()
```

7.3. Eliminar eventos aleatoriamente

Estas funciones permiten eliminar eventos de un patrón según una probabilidad determinada.

Función	Descripción	Ejemplo en Strudel	Aplicación
`degradeBy(amount)`	Elimina eventos aleatoriamente según una probabilidad entre 0 y 1.	`s("hh*8").degradeBy(0.2)`	Elimina eventos con una probabilidad del 20 %.
`degrade()`	Elimina aleatoriamente el 50 % de los eventos. Es equivalente a `degradeBy(0.5)`.	`s("hh*8").degrade()`	Genera una versión menos densa del patrón.
`undegradeBy(amount)`	Funciona de manera inversa a `degradeBy()`: deja pasar los eventos que serían eliminados por la versión	`s("hh*8").undegradeBy(0.2)`	Permite obtener la selección complementaria de eventos.

Función	Descripción	Ejemplo en Strudel	Aplicación
	correspondiente de `degradeBy()`.		
`undegrade()`	Versión inversa de `degrade()`. Deja pasar los eventos que la versión normal eliminaría.	`s("hh*8").undegrade()`	Genera el patrón complementario al de `degrade()`.

En `degradeBy()`, el valor indica la probabilidad de eliminar cada evento:

- `0`: no se elimina ningún evento.
- `0.2`: cada evento tiene un 20 % de probabilidad de desaparecer.
- `0.5`: cada evento tiene un 50 % de probabilidad de desaparecer.
- `1`: se eliminan todos los eventos.

Ejemplo:

```
s("hh*8").degradeBy(0.2)
```

También se puede expresar mediante Mini Notation:

```
s("[hh?0.2]*8")
```

Una aplicación interesante es combinar `degradeBy()` y `undegradeBy()` para separar los eventos complementarios entre dos capas:

```
s("hh*10").layer(
  x => x.degradeBy(0.2).pan(0),
  x => x.undegradeBy(0.8).pan(1)
)
```

7.4. Aplicar transformaciones con una probabilidad

También podemos aplicar una función a un patrón solamente algunas veces.

Estas funciones reciben:

- Una probabilidad entre 0 y 1 (cuando corresponde).
- Una función que contiene la transformación que queremos aplicar.

Función	Descripción	Probabilidad	Ejemplo en Strudel
`sometimesBy(p, fn)`	Aplica una transformación aleatoriamente con la probabilidad indicada.	La indicada en `p`.	`s("hh*8").sometimesBy(0.4,x=>x.spe`

Función	Descripción	Probabilidad	Ejemplo en Strudel
`sometimes(fn)`	Aplica una transformación con una probabilidad del 50 %.	50 %	`s("hh*8").sometimes(x=>x.speed("0`
`someCyclesBy(p, fn)`	Aplica una transformación aleatoriamente en determinados ciclos, según la probabilidad indicada.	La indicada en `p`.	`s("bd,hh*8").someCyclesBy(0.3,x=>`
`someCycles(fn)`	Versión abreviada de `someCyclesBy(0.5, fn)`.	50 % por ciclo.	`s("bd,hh*8").someCycles(x=>x.speed`

La diferencia entre `sometimesBy()` y `someCyclesBy()` es que la primera aplica la decisión aleatoria al patrón, mientras que la segunda toma la decisión ciclo por ciclo.

7.5. Atajos para probabilidades habituale

Strudel incluye funciones abreviadas para no tener que indicar manualmente una probabilidad en cada ocasión.

Función	Equivalencia	Probabilidad	Ejemplo
`often(fn)`	`sometimesBy(0.75, fn)`	75 %	`s("hh*8").often(x=>x.speed("0.5")`
`rarely(fn)`	`sometimesBy(0.25, fn)`	25 %	`s("hh*8").rarely(x=>x.speed("0.5"`
`almostNever(fn)`	`sometimesBy(0.1, fn)`	10 %	`s("hh*8").almostNever(x=>x.speed(`
`almostAlways(fn)`	`sometimesBy(0.9, fn)`	90 %	`s("hh*8").almostAlways(x=>x.speed`
`never(fn)`	`sometimesBy(0, fn)`	0 %	`s("hh*8").never(x=>x.speed("0.5")`
`always(fn)`	`sometimesBy(1, fn)`	100 %	`s("hh*8").always(x=>x.speed("0.5"`

Idea fundamental: los modificadores aleatorios permiten que un patrón no sea exactamente igual en cada reproducción. Son útiles para introducir variaciones, evitar repeticiones demasiado mecánicas y generar ritmos o melodías parcialmente impredecibles.

---

8. Conditional Modifiers

Documentación de Conditional Modifiers

Los *conditional modifiers* permiten aplicar transformaciones a un patrón solamente cuando se cumple una determinada condición.

Las condiciones pueden depender del número de ciclo, de una estructura binaria o de la posición de los eventos.
También permiten dividir patrones, seleccionar partes de ellos, modificar acordes y controlar cuándo se reinician determinadas secuencias.

8.1. Aplicar transformaciones según el ciclo

Función	Descripción	Ejemplo en Strudel	Aplicación
`firstOf(n, fn)`	Aplica una transformación al principio de cada grupo de n ciclos.	`note("c3 d3 e3 g3").firstOf(4,x=>x.rev())`	Modifica el primer ciclo de cada grupo de cuatro.
`lastOf(n, fn)`	Aplica una transformación al último ciclo de cada grupo de n ciclos.	`note("c3 d3 e3 g3").lastOf(4,x=>x.rev())`	Modifica el último ciclo de cada grupo de cuatro.
`when(pattern, fn)`	Aplica una transformación cuando el patrón indicado tiene un estado verdadero.	`"c3 eb3 g3".when("<0 1>/2",x=>x.sub("5")).note()`	Condiciona una transformación a otro patrón.

Por ejemplo:

```
note("c3 d3 e3 g3")
  .firstOf(4, x => x.rev())
```

El patrón se invierte al principio de cada grupo de cuatro ciclos.
En cambio, con `lastOf(4, ...)`, la inversión se aplica al último ciclo del grupo.

8.2. Dividir un patrón en partes

Estas funciones permiten dividir un patrón en varias secciones y aplicar transformaciones a cada una.

Función	Sinónimo	Descripción	Ejemplo en Strudel
`chunk(n, fn)`	`slowChunk()` / `slowchunk()`	Divide el patrón en n partes y aplica la transformación a cada parte, recorriéndolas en orden. Se utiliza una parte por ciclo.	`"0 1 2 3".chunk(4,x=>x.add(7)).scale("A:minor"`
`chunkBack(n, fn)`	`chunkback()`	Funciona como `chunk()`, pero recorre las	`"0 1 2 3".chunkBack(4,x=>x.add(7)).scale("A:mi`

Función	Sinónimo	Descripción	Ejemplo en Strudel
		partes en orden inverso.	
`fastChunk(n, fn)`	`fastchunk()`	Divide el patrón y aplica las transformaciones a sus partes sin repetir los ciclos del patrón original para cada conjunto de partes.	`"<0 8> 1 2 3 4 5 6 7".fastChunk(4,x=>x.color("red")).slow(`

Ejemplo:

```
"0 1 2 3"
  .chunk(4, x => x.add(7))
  .scale("A:minor")
  .note()
```

El patrón se divide en cuatro partes. En cada ciclo se aplica la transformación a una de esas partes, sumando siete a sus valores.

8.3. Seleccionar notas de un acorde

Las funciones `arp()` y `arpWith()` permiten seleccionar notas individuales dentro de un conjunto de notas simultáneas.

Función	Descripción	Ejemplo en Strudel
`arp(pattern)`	Selecciona índices concretos de las notas de un acorde o conjunto de notas apiladas.	`note("<[c,eb,g]!2 [c,f,ab] [d,f,ab]>").arp("0 [0,2] 1 [0,2]")`
`arpWith(fn)`	Permite seleccionar notas mediante una función que recibe los eventos del acorde.	`note("<[c,eb,g]!2 [c,f,ab] [d,f,ab]>").arpWith(haps=>haps[2])`

Por ejemplo, si tenemos un acorde formado por Do, Mi bemol y Sol, `arp("0")` selecciona la primera nota, `arp("1")` la segunda y `arp("2")` la tercera.

Esto permite transformar acordes en secuencias de notas, creando arpegios.

8.4. Controlar la estructura de un patrón

Función	Descripción	Ejemplo en Strudel	Aplicación
`struct(pattern)`	Aplica una estructura a un patrón, indicando dónde se reproducen	`note("c,eb,g").struct("x ~ x ~ ~ x ~ x ~ ~ ~ x ~ x ~ ~").slow(2)`	Controla el ritmo de reproducción de un patrón.

Función	Descripción	Ejemplo en Strudel	Aplicación
	los eventos y dónde hay silencios.		
`mask(pattern)`	Devuelve silencio cuando la máscara contiene un `0` o un `~`.	`note("c [eb,g] d [eb,g]").mask("<1 [0 1]>")`	Permite activar o silenciar eventos según una estructura externa.
`invert()`	Invierte los valores de una estructura binaria, intercambiando los `1` y los `0`.	`s("bd").struct("1 0 0 1 0 0 1 0".lastOf(4,invert))`	Invierte una máscara rítmica. También tiene el sinónimo `inv()`.

En `struct()`, el símbolo `x` representa una posición en la que se reproduce el evento, mientras que `~` representa una posición silenciosa.

8.5. Reiniciar patrones

Estas funciones permiten controlar cuándo una secuencia vuelve a empezar.

Función	Descripción	Ejemplo en Strudel
`reset(pattern)`	Reinicia el patrón al principio de su ciclo actual cada vez que aparece un evento en el patrón de reinicio.	`s("[<bd lt> sd]*2, hh*8").reset("<x@3 x(5,8)>")`
`restart(pattern)`	Reinicia el patrón desde el ciclo 0 cada vez que aparece un evento en el patrón de reinicio.	`s("[<bd lt> sd]*2, hh*8").restart("<x@3 x(5,8)>")`
`hush()`	Silencia un patrón.	`stack(s("bd").hush(),s("hh*3"))`

La diferencia entre `reset()` y `restart()` es importante:

- `reset()` vuelve al comienzo del ciclo actual.
- `restart()` vuelve al comienzo del patrón, es decir, al ciclo 0.

8.6. Seleccionar patrones mediante índices

Estas funciones permiten elegir entre distintos patrones utilizando otro patrón como selector.
Por ejemplo, podemos tener una secuencia de números que indique qué elemento de una lista queremos reproducir.

Función	Descripción	Ejemplo o aplicación
`pick(pattern, xs)`	Selecciona patrones o valores de una lista mediante índices o nombres. Mantiene la estructura del patrón original.	`note("<0 1 2!2 3>".pick(["g a","e f","f g f g","g c d"]))`

Función	Descripción	Ejemplo o aplicación
`pickmod(pattern, xs)`	Funciona como `pick()`, pero si el índice supera el tamaño de la lista, vuelve al principio mediante un comportamiento circular.	El quinto elemento de una lista de tres selecciona el segundo.
`pickF(pattern, functions)`	Selecciona qué función aplicar a otro patrón utilizando índices o nombres.	`s("bd [rim hh]").pickF("<0 1 2>",[rev,jux(rev),fast(2)])`
`pickmodF(pattern, functions)`	Funciona como `pickF()`, pero los índices superiores al tamaño de la lista vuelven al principio.	Permite seleccionar funciones de manera circular.
`pickRestart(pattern, xs)`	Selecciona un patrón y lo reinicia cuando se activa su índice.	Útil cuando queremos que cada selección comience desde el principio.
`pickmodRestart(pattern, xs)`	Versión circular de `pickRestart()`.	Reinicia el patrón seleccionado y permite que los índices vuelvan al principio.
`pickReset(pattern, xs)`	Selecciona un patrón y lo resetea cuando se activa su índice.	Permite controlar el punto de reinicio de cada selección.
`pickmodReset(pattern, xs)`	Versión circular de `pickReset()`.	Combina selección circular y reinicio.

Ejemplo de `pick()`:

```
note("<0 1 2!2 3>".pick([
  "g a",
  "e f",
  "f g f g",
  "g c d"
]))
```

En este caso, los números del patrón seleccionan los elementos correspondientes de la lista.

Ejemplo de `pickF()`:

```
s("bd [rim hh]")
  .pickF("<0 1 2>", [
    rev,
    jux(rev),
    fast(2)
  ])
```

En este ejemplo, el patrón selector determina qué transformación se aplica: invertir, aplicar `jux(rev)` o acelerar.

8.7. Seleccionar patrones manteniendo o comprimiendo su estructura

También existen funciones que seleccionan patrones de una lista, pero modifican cómo se ajustan temporalmente al patrón que los selecciona.

Función	Sinónimo	Descripción	Ejemplo en Strudel
`inhabit(pattern, xs)`	`pickSqueeze()`	Selecciona patrones o valores de una lista o tabla de nombres. A diferencia de `pick()`, comprime los ciclos seleccionados para ajustarlos al patrón que los contiene.	`"<a b [a,b]>".inhabit({a:s("bd(3,8)"), sd")})`
`inhabitmod(pattern, xs)`	`pickmodSqueeze()`	Funciona como `inhabit()`, pero los índices superiores al tamaño de la lista vuelven al principio.	Permite seleccionar patrones de for
`squeeze(pattern, xs)`	—	Selecciona un patrón o valor mediante índices y comprime el patrón seleccionado para ajustarlo a la duración del evento que lo selecciona.	`note(squeeze("<0@2 [1!2] 2>",[" g f g","g a c d"]))`

Idea fundamental: los modificadores condicionales permiten controlar cuándo y cómo se transforma un patrón. Son útiles para crear variaciones estructuradas, arpegios, cambios por ciclo, máscaras rítmicas y secuencias que se reinician o seleccionan de forma controlada.

---

9. Accumulation Modifiers

Documentación de Accumulation Modifiers

Los *accumulation modifiers* permiten combinar un patrón original con versiones transformadas de ese mismo patrón.
En lugar de modificar únicamente la secuencia original, estas funciones pueden superponer otras versiones, crear capas, desplazar eventos en el tiempo o generar repeticiones similares a un eco.
Son especialmente útiles para crear armonías, duplicar melodías, añadir capas y generar efectos rítmicos.

9.1. Superponer y crear capas

Función	Descripción	Ejemplo en Strudel
`superimpose(fn)`	Superpone el resultado de aplicar una función al patrón original. Se reproducen tanto el patrón original como su versión transformada.	`"0 2 4 6".superimpose(x=>x.add(2)).scale("C:minor").note()`
`layer(fn, ...)`	Crea capas con los resultados de las funciones indicadas. A diferencia de `superimpose()`, no conserva automáticamente el patrón original.	`"0 2 4 6".layer(x=>x.add("0,2")).scale("C:minor").note()`

Ejemplo de `superimpose()`:

```
"<0 2 4 6 ~ 4 ~ 2 0!3 ~!5>*8"
  .superimpose(x => x.add(2))
  .scale("C minor")
  .note()
```

El patrón original se reproduce junto con otra versión en la que se han sumado dos unidades a sus valores. Al aplicar la escala, ambas versiones se convierten en notas musicales.

Ejemplo de `layer()`:

```
"<0 2 4 6 ~ 4 ~ 2 0!3 ~!5>*8"
  .layer(x => x.add("0,2"))
  .scale("C minor")
  .note()
```

En este caso se construyen capas a partir del patrón, pero no se conserva automáticamente una capa original independiente como ocurre con `superimpose()`.

9.2. Desplazar patrones en el tiempo

Función	Descripción	Parámetros	Ejemplo en Strudel
`off(time, fn)`	Superpone el resultado de una transformación sobre el patrón original, pero desplazándolo temporalmente.	`time`: desplazamiento temporal. `fn`: función que se aplica al patrón.	`"c3 eb3 g3".off(1/8,x=>x.add(7)).note()`

Ejemplo:

```
"c3 eb3 g3"
  .off(1/8, x => x.add(7))
  .note()
```

Se reproduce el patrón original y, además, una segunda versión transpuesta siete unidades y desplazada temporalmente una octava parte de ciclo.

9.3. Crear ecos y repeticiones

Función	Descripción	Parámetros	Ejemplo en Strudel
`echo(times,time,feedback)`	Superpone varias repeticiones del patrón, desplazándolas en el tiempo y reduciendo progresivamente su velocidad o intensidad mediante el parámetro de feedback.	`times`: número de repeticiones. `time`: desplazamiento entre repeticiones. `feedback`: multiplicador de velocidad de cada repetición.	`s("bd sd").echo(3,1/6,0.8)`
`echoWith(times,time,fn)`	Crea varias superposiciones desplazadas en el tiempo y aplica una transformación diferente en cada repetición.	`times`: número de repeticiones. `time`: desplazamiento temporal. `fn`: función aplicada a cada repetición, que recibe el patrón y el índice de iteración.	`"<0 [2 4]>".echoWith(4,1/8 (p,n)=>p.add(n*2)).scale("`
`stut(times,feedback,time)`	Versión antigua de `echo()`, con	`times`: número de	`s("bd sd").stut(3,0.8,1/6)`

Función	Descripción	Parámetros	Ejemplo en Strudel
	los dos últimos parámetros en un orden diferente.	repeticiones. `feedback`: multiplicador. `time`: desplazamiento temporal.	

Ejemplo de `echo()`:

```
s("bd sd").echo(3,1/6,0.8)
```

Se generan tres repeticiones superpuestas, separadas por un sexto de ciclo. El parámetro `0.8` controla el multiplicador aplicado progresivamente a la velocidad o intensidad de las repeticiones.

Ejemplo de `echoWith()`:

```
"<0 [2 4]>"
  .echoWith(4,1/8,(p,n)=>p.add(n*2))
  .scale("C:minor")
  .note()
```

En este caso, cada repetición recibe una transformación distinta. La función utiliza `n` como índice de la iteración y suma una cantidad diferente en cada repetición.

Importante: `stut()` está marcado como obsoleto (*deprecated*) en la documentación. Para código nuevo es preferible utilizar `echo()`.

Idea fundamental: los modificadores de acumulación permiten crear nuevas capas musicales a partir de un patrón existente. En lugar de escribir varias secuencias independientes, podemos generar versiones transformadas, superponerlas y desplazarlas temporalmente.

---

10. Tonal Functions

Documentación de Tonal Functions

Las *tonal functions* son funciones que permiten realizar operaciones musicales relacionadas con notas, escalas, acordes y armonía.

Strudel utiliza la biblioteca **Tonal.js** para proporcionar estas herramientas.
Permiten trabajar con grados de una escala, transponer melodías, convertir símbolos de acordes en notas y generar acompañamientos.

10.1. Convertir acordes en posiciones musicales: `voicing()`

La función `voicing()` convierte símbolos de acordes en disposiciones concretas de notas.

Por ejemplo, un acorde de Do mayor puede representarse mediante diferentes posiciones u ordenaciones de sus notas. La función permite elegir cómo se distribuyen esas notas.

```
n("0 1 2 3")
  .chord("<C Am F G>")
  .voicing()
```

En este ejemplo se define una secuencia de acordes y se utiliza `voicing()` para obtener sus posiciones musicales.

La función admite diferentes parámetros de control:

Parámetro	Descripción
`chord`	Acorde que se quiere convertir. Se indica mediante una nota seguida del símbolo del acorde, por ejemplo `C`, `Am`, `G7` o `Bb^7`. Es el único parámetro obligatorio.
`dict`	Diccionario de posiciones de acordes que se quiere utilizar. Si no se indica, se utiliza el diccionario predeterminado.
`anchor`	Nota utilizada como referencia para colocar el acorde.
`mode`	Determina cómo se alinea el acorde respecto a la nota de referencia.
`offset`	Número entero que desplaza la posición del acorde hacia arriba o hacia abajo hasta otra posición.
`n`	Si se especifica, el voicing se reproduce como si fuera una escala. Si se supera el rango de notas, se cambia de octava.

Los modos disponibles para `mode` son:

Modo	Descripción
`below`	La nota más aguda del acorde debe ser menor o igual que la nota de referencia.
`duck`	La nota más aguda debe ser menor que la nota de referencia, sin incluirla.
`above`	La nota más grave debe ser mayor o igual que la nota de referencia.
`root`	La nota más grave debe ser la nota fundamental más baja del acorde que sea mayor o igual que la referencia.
`oldabove`	Comportamiento antiguo de `above`, conservado por compatibilidad.
`oldroot`	Comportamiento antiguo de `root`, conservado por compatibilidad.

Todos estos parámetros son opcionales, excepto el acorde.

10.2. Trabajar con escalas: `scale()`

La función `scale()` convierte números en notas musicales utilizando una escala determinada. También permite ajustar notas existentes para que pertenezcan a una escala.

```
n("0 2 4 6 4 2")
  .scale("C:major")
```

En este ejemplo, los números representan grados de la escala de Do mayor.

La numeración comienza en cero:

- `0`: primer grado.
- `1`: segundo grado.
- `2`: tercer grado.
- `3`: cuarto grado.
- Y así sucesivamente.

La escala se escribe indicando primero la nota fundamental y después el tipo de escala, separados por `:`.
Ejemplos:

Expresión	Significado
`C:major`	Escala de Do mayor.
`C:minor`	Escala de Do menor.
`F#:major`	Escala de Fa sostenido mayor.
`Bb:minor`	Escala de Si bemol menor.
`C:major:blues`	Escala de Do mayor blues.

Si no se indica una octava, Strudel utiliza por defecto la octava 3.

También se pueden utilizar números negativos para desplazarse hacia atrás en la escala. Además, se pueden utilizar sostenidos (`#`) y bemoles (`b`) para representar notas fuera de la escala.
Por ejemplo:

```
n("[0,7] 4 [2,7] 4")
  .scale("C:<major minor>/2")
  .s("piano")
```

En este caso se alterna entre las escalas de Do mayor y Do menor.

10.3. Transponer notas: `transpose()`

La función `transpose(semitones)` permite desplazar todas las notas de un patrón una cantidad determinada de semitonos.

```
note("c3 e3 g3")
  .transpose(12)
```

En este ejemplo, todas las notas suben doce semitonos, es decir, una octava.

También se pueden utilizar intervalos expresados mediante notación musical científica.

10.4. Transponer dentro de una escala: `scaleTranspose()`

La función `scaleTranspose(steps)` permite transponer notas utilizando pasos de una escala, en lugar de semitonos.

```
"[-8 [2,4,6]]*2"
  .scale("C4 bebop major")
  .scaleTranspose("<0 -1 -2 -3 -4 -5 -6 -4>*2")
  .note()
```

La diferencia respecto a `transpose()` es:

- `transpose()` desplaza las notas una cantidad fija de semitonos.
- `scaleTranspose()` desplaza las notas una cantidad de grados dentro de la escala establecida.

Por tanto, si utilizamos una escala, podemos modificar una melodía manteniendo su relación con esa escala.

10.5. Obtener las notas fundamentales de los acordes: `rootNotes()`

La función `rootNotes(octave = 2)` convierte los símbolos de acordes en sus notas fundamentales, colocándolas en la octava indicada.

```
"<C^7 A7b13 Dm7 G7>*2"
  .rootNotes(2)
  .note()
```

Por ejemplo, si tenemos una progresión de acordes, `rootNotes()` permite obtener las notas fundamentales de cada uno para construir una línea de bajo.

También se puede combinar con otras funciones para crear un acompañamiento:

```
"<C^7 A7b13 Dm7 G7>*2".layer(
  x => x.voicings("lefthand")
    .struct("[~ x]*2")
    .note(),

  x => x.rootNotes(2)
    .note()
    .s("sawtooth")
    .cutoff(800)
)
```

En este ejemplo se generan dos capas:

1. Una capa de acordes utilizando `voicings("lefthand")` y una estructura rítmica.
2. Una línea de bajo utilizando las notas fundamentales de los acordes, con un sonido de diente de sierra y un filtro.

Idea fundamental: las funciones tonales permiten trabajar con conceptos musicales de mayor nivel de abstracción. En lugar de tener que indicar todas las notas de un acorde o calcular manualmente los intervalos, podemos definir escalas, acordes y transposiciones mediante funciones.

11. REPL: funcionamiento interno de Strudel

Documentación de REPL

El **REPL** (*Read, Evaluate, Print/Play, Loop*) es el entorno interactivo de live coding de Strudel. Permite escribir y modificar código mientras la música se está reproduciendo.
Aunque Strudel también puede utilizarse como una biblioteca dentro de cualquier proyecto JavaScript, su interfaz principal es un editor que funciona en el navegador.

Su objetivo es permitir que el usuario modifique patrones musicales en tiempo real y pueda escuchar inmediatamente los cambios.

El REPL incluye:

- Un editor de código basado en **CodeMirror**.
- Controles para iniciar y detener la reproducción.
- Información sobre la reproducción y los patrones.
- Retroalimentación visual que resalta las partes del código que están generando los eventos que suenan en ese momento.
- Diferentes sistemas de salida para generar audio u otros tipos de resultados

11.1. Flujo general de funcionamiento

El funcionamiento del REPL se puede dividir en tres fases principales:

Fase	Qué ocurre	Resultado
1. Escritura y evaluación del código	El usuario escribe o modifica código. Strudel lo transforma y lo evalúa.	Se obtiene un objeto `Pattern`.
2. Planificación de eventos (*Scheduling*)	El `Scheduler` consulta periódicamente el patrón activo para obtener los eventos del siguiente intervalo temporal.	Se generan los eventos musicales que deben reproducirse.
3. Ejecución de eventos	Cada evento se envía a la salida correspondiente mediante el método `onSchedule()`.	Se produce el sonido u otro resultado asociado al evento.

El flujo completo sería:

```
Usuario escribe código
        |
        v
Transpilación del código
        |
        v
Evaluación del código
        |
        v
Objeto Pattern
        |
        v
Scheduler consulta el patrón
        |
        v
Generación de Events (Haps)
```

```
        |
        v
onSchedule()
        |
        v
Salida de audio
```

Es importante distinguir entre el patrón y los eventos: el patrón describe qué debe ocurrir y cuándo, mientras que el scheduler obtiene los eventos concretos que corresponden a cada intervalo de tiempo.

11.2. Transpilación y evaluación del código

Para convertir el código escrito por el usuario en un patrón musical, Strudel realiza dos operaciones:

1. **Transpilación:** transforma el código original para adaptarlo al funcionamiento del REPL.
2. **Evaluación:** ejecuta el código transformado para obtener un objeto `Pattern`.

La transpilación permite simplificar la sintaxis que utiliza el usuario.
Por ejemplo, en Strudel podemos escribir:

```
note("c3 [e3 g3]*2")
```

Sin embargo, internamente esta expresión se transforma en:

```
note(m('c3 [e3 g3]*2', 5))
```

La función `m()` interpreta la cadena como Mini Notation y genera un patrón.

El segundo parámetro contiene la posición del código original donde se encuentra la cadena. Esta información permite que Strudel pueda resaltar visualmente las partes de la notación que están reproduciéndose.

Herramientas utilizadas

Herramienta	Función
`acorn`	Analiza el código JavaScript y genera un AST (*Abstract Syntax Tree*).
`AST`	Representa la estructura del código mediante un árbol de nodos que se puede analizar y modificar.
`escodegen`	Genera el código JavaScript transformado a partir del AST.

El proceso es:

1. El usuario escribe código JavaScript.
2. `acorn` analiza ese código y genera un AST.
3. Strudel modifica la estructura del AST para introducir las transformaciones necesarias.
4. `escodegen` genera el código JavaScript resultante.

5. El código transformado se evalúa y produce un objeto `Pattern`.
   Importante: la transpilación no genera directamente el sonido. Su función es preparar el código para que pueda ejecutarse y convertirse en un patrón.

11.3. Procesamiento de Mini Notation

La Mini Notation permite representar patrones musicales de forma compacta. Para interpretarla, Strudel utiliza un parser independiente del parser de JavaScript.
El proceso es el siguiente:

1. Se recibe una cadena escrita en Mini Notation.
2. El parser analiza su sintaxis.
3. Se genera un AST que representa la estructura del patrón.
4. Ese AST se utiliza para construir un patrón mediante las funciones habituales de Strudel.

Herramientas utilizadas

Herramienta	Función
`PEG`	Formalismo utilizado para definir la gramática de un lenguaje.
`peggy`	Herramienta que genera parsers a partir de una gramática PEG.
`mini package`	Paquete donde está implementada la gramática de Mini Notation.
`AST`	Representación estructurada de la notación, que permite construir el patrón.

La implementación de Mini Notation está basada en el proyecto `krill`.

Ejemplo de transformación

Partimos de la siguiente expresión:

```
"c3 [e3 g3]"
```

El parser genera una estructura de árbol que representa:

- Un evento `c3`.
- Un grupo que contiene los eventos `e3` y `g3`.

Esta estructura se traduce en llamadas a funciones de Strudel:

```
seq(
  reify('c3').withLoc(6, 9),
  seq(
    reify('e3').withLoc(10, 12),
    reify('g3').withLoc(13, 15)
  )
)
```

En este ejemplo:

- `seq()` construye una secuencia de patrones.
- `reify()` convierte los elementos en patrones.
- `withLoc()` almacena la posición del elemento dentro del código original.

Idea fundamental: Mini Notation no se interpreta directamente como sonido. Primero se convierte en una estructura de datos y después esa estructura se utiliza para construir un patrón.

11.4. Información de posición y resaltado visual

Strudel almacena la posición de los elementos dentro del código original para poder resaltar visualmente los eventos que se están reproduciendo.
Para ello utiliza `withLoc()`.

La posición de cada elemento se calcula combinando dos valores:

Información	Descripción
Posición inicial de la cadena	Lugar donde comienza la cadena de Mini Notation dentro del código JavaScript.
Posición interna del elemento	Lugar que ocupa el elemento dentro de la propia cadena de Mini Notation.

La suma de ambas posiciones permite obtener la ubicación real del elemento dentro del editor.

Por ejemplo, si una cadena empieza en la posición 6 del código y una nota ocupa las posiciones 0 a 3 dentro de esa cadena, se puede determinar dónde debe resaltarse esa nota en el editor.

Esto permite que Strudel identifique qué partes del código están generando los eventos que suenan en cada momento.

Aplicación: esta información es especialmente útil en un entorno de live coding porque conecta visualmente el código con la música que se está reproduciendo.

11.5. Planificación de eventos (Scheduling)

Una vez que el código se ha evaluado y se ha obtenido un objeto `Pattern`, el scheduler se encarga de consultar ese patrón y obtener los eventos que deben reproducirse.

El scheduler funciona mediante consultas periódicas.

En cada intervalo:

1. Consulta el patrón activo.
2. Obtiene los eventos correspondientes al intervalo temporal actual.

3. Avanza el tiempo.
4. Envía cada evento a la función encargada de programar su reproducción.

En Strudel, los eventos también reciben el nombre de **Haps**.

Conceptos importantes

Concepto	Descripción
`Pattern`	Objeto que representa la estructura musical y permite consultar qué eventos ocurren en un intervalo de tiempo.
`Scheduler`	Componente que consulta periódicamente el patrón y organiza la reproducción de los eventos.
`Event` / `Hap`	Evento concreto generado a partir del patrón, con información temporal y valores asociados.
`queryArc(start, end)`	Consulta los eventos que pertenecen a un intervalo temporal determinado.
`onSchedule()`	Método utilizado por la salida para programar la ejecución de cada evento.

Ejemplo simplificado de un scheduler

La documentación presenta una implementación simplificada para explicar cómo funciona:

```
let pattern = seq('c3', ['e3', 'g3']);

let interval = 0.5;
let time = 0;
let minLatency = 0.1;

setInterval(() => {
  const haps = pattern.queryArc(time, time + interval);

  time += interval;

  haps.forEach((hap) =>
    onSchedule(hap, hap.whole.begin + minLatency)
  );

}, interval * 1000);
```

En este ejemplo:

Variable	Significado
`pattern`	Patrón que se está reproduciendo.
`interval`	Duración del intervalo temporal que se consulta, en segundos.
`time`	Comienzo del intervalo actual.
`minLatency`	Tiempo mínimo que debe transcurrir antes de que un evento se reproduzca.
`haps`	Lista de eventos obtenidos mediante `queryArc()`.

La función `queryArc()` recibe el principio y el final del intervalo y devuelve los eventos que deben ocurrir durante ese periodo.

Después, cada evento se envía a `onSchedule()` para programar su reproducción.

Actualización de patrones en tiempo real

Una característica importante es que `Pattern.queryArc()` es una función pura: dado un mismo patrón y un mismo intervalo temporal, devuelve el mismo conjunto de eventos.

Esto permite consultar el patrón utilizando intervalos de diferentes tamaños sin modificar el resultado musical.

También permite sustituir el patrón activo mientras el scheduler sigue funcionando.

Cuando el usuario modifica el código:

- Se evalúa el nuevo código.
- Se obtiene un nuevo objeto `Pattern`.
- El scheduler utiliza el nuevo patrón en su siguiente consulta.
- El reloj de reproducción continúa funcionando.

Por tanto, no es necesario detener y volver a iniciar toda la reproducción cada vez que se modifica el código.

Latencia

La latencia es el tiempo que transcurre entre la evaluación de un cambio y el momento en el que ese cambio se escucha.

En el ejemplo simplificado, depende del intervalo de consulta y de la latencia mínima.

La documentación indica que la implementación actual de Strudel utiliza:

Parámetro	Valor
Intervalo de consulta	50 ms
Latencia mínima	100 ms
Latencia resultante	Entre 50 y 150 ms

La implementación real también debe tener en cuenta que `setInterval()` no garantiza una ejecución perfectamente precisa. Por eso, el ejemplo anterior es únicamente una simplificación del funcionamiento real.

11.6. Parámetros de control (Control Parameters)

Los parámetros de control permiten modificar diferentes propiedades de los eventos musicales.

En lugar de que cada evento contenga únicamente una nota, puede contener varios valores que determinan cómo debe reproducirse.
Por ejemplo:

```
note('c3 e3')
  .cutoff(1000)
  .s('sawtooth')
  .queryArc(0, 1)
  .map((hap) => hap.value);
```

El resultado simplificado es:

```
[
  { note: 'c3', cutoff: 1000, s: 'sawtooth' },
  { note: 'e3', cutoff: 1000, s: 'sawtooth' }
]
```

Cada evento contiene tres propiedades:

Parámetro	Valor	Función
`note`	`c3` o `e3`	Indica la nota que se reproduce.
`cutoff`	`1000`	Indica la frecuencia de corte del filtro.
`s`	`sawtooth`	Indica el sonido utilizado.

Las funciones de parámetros de control permiten añadir o modificar propiedades dentro del objeto de valores de cada evento.

Tipos de valores aceptados

Cada función de control puede recibir:

Tipo de entrada	Descripción	Ejemplo
Valor primitivo	Un valor individual, como un número o una cadena.	`.cutoff(1000)`
Lista de valores	Una lista que se convierte en un patrón secuencial.	`.cutoff([500,1000,2000])`
`Pattern`	Otro patrón cuyos valores controlan el parámetro.	`.cutoff(sine.range(500,4000))`

Strudel incluye un conjunto amplio de parámetros de control predeterminados, basados en los utilizados por Tidal y SuperDirt.
Estos parámetros utilizan nombres relacionados con la música y el audio, como `note`, `s`, `gain` o `cutoff`.

Crear parámetros de control personalizados

También es posible crear parámetros propios mediante `createParams()`.

```
const { x, y } = createParams('x', 'y');

x(sine.range(0, 200))
  .y(cosine.range(0, 200));
```

En este ejemplo:

1. Se crean dos parámetros personalizados: `x` e `y`.
2. El parámetro `x` recibe los valores de una señal sinusoidal.
3. El parámetro `y` recibe los valores de una señal cosenoidal.
4. Ambos parámetros pueden utilizarse para representar las coordenadas de un círculo.

Esto demuestra que los parámetros de control no tienen por qué estar relacionados exclusivamente con el audio. También pueden representar cualquier propiedad que necesite cambiar con el tiempo.

11.7. Sistemas de salida (Outputs)

Una vez que el scheduler ha generado los eventos y estos contienen sus parámetros de control, el sistema de salida se encarga de convertirlos en un resultado real.

La salida predeterminada de Strudel es **WebAudio**, que utiliza las capacidades de audio del navegador para generar sonido.
El scheduler llama a `onSchedule()` para que la salida pueda programar cada evento.

```
function onSchedule(hap, time, cps) {
  const { note } = hap.value;

  const o = getAudioContext().createOscillator();

  o.frequency.value = getFreq(note);

  o.start(time);
  o.stop(time + hap.duration / cps);

  o.connect(getAudioContext().destination);
}
```

Qué hace cada operación

Operación	Descripción
`hap.value`	Contiene los parámetros asociados al evento.
`note`	Se extrae la nota del evento.
`getAudioContext()`	Obtiene el contexto de audio utilizado por WebAudio.
`createOscillator()`	Crea un oscilador para generar sonido.
`getFreq(note)`	Convierte la nota musical en una frecuencia.

Operación	Descripción
`o.start(time)`	Programa el inicio del sonido en un momento determinado.
`o.stop(...)`	Programa el final del sonido.
`o.connect(...)`	Conecta el oscilador con la salida de audio.

El ejemplo crea un oscilador por cada evento y utiliza la nota para determinar su frecuencia.

La implementación real de Strudel es más compleja: utiliza muchos más parámetros para controlar sintetizadores, samples y efectos.

Separación entre patrón y salida

Una de las ideas más importantes de la arquitectura de Strudel es que el patrón no necesita saber cómo se va a reproducir.

El patrón contiene los eventos y sus valores, mientras que la salida decide qué hacer con ellos.
Por ejemplo, un mismo evento podría utilizarse para:

- Generar audio mediante WebAudio.
- Controlar un sintetizador.
- Reproducir samples.
- Enviar información a otro sistema.
  Esto permite separar la lógica musical de la tecnología encargada de producir el sonido.

11.8. Resumen de la arquitectura

Componente	Responsabilidad	Entrada	Salida
Editor (CodeMirror)	Permite escribir y modificar el código.	Código del usuario.	Código actualizado.
Transpilador	Transforma el código para adaptarlo al REPL.	Código JavaScript.	Código JavaScript transformado.
Parser de Mini Notation	Interpreta la sintaxis musical compacta.	Cadena de Mini Notation.	AST de Mini Notation.
Evaluador	Ejecuta el código transformado.	Código JavaScript transformado.	Objeto `Pattern`.
`Pattern`	Representa la estructura musical y sus eventos.	Funciones y datos del patrón.	Eventos para un intervalo temporal.
`Scheduler`	Consulta el patrón y organiza la reproducción.	Patrón activo y tiempo.	Eventos programados.
Control Parameters	Define las propiedades de cada evento.	Valores y patrones de control.	Objeto de valores del evento.
Output	Convierte los eventos en un resultado real.	Eventos y parámetros.	Audio u otra salida.